

Design and Implementation of a Scalable Multi-User Database System for Smart Healthcare Management

Serhet Gökdemir
21058024

2026

Contents

1	Requirements Analysis	1
1.1	Introduction	1
1.2	User Roles and Permissions	1
1.3	Core Entities	1
1.4	Core Relationships	1
1.5	Constraints and Business Rules	2
2	ER and EER Models	2
2.1	ER Diagram	2
2.2	Enhanced ER (EER) Diagram	2
2.2.1	Explanation of EER Concepts	3
3	Mapping to Relational Model	3
3.1	Relational Schema	3
3.2	Mapping Rules Applied	7
4	Functional Dependencies and Normalization	7
4.1	Functional Dependencies (FDs)	7
4.2	Normalization Steps	10
5	SQL Implementation and Advanced Queries	14
5.1	Data Insertion	14
5.2	Advanced SQL Queries	14
5.2.1	Query Outputs	18
6	Relational Algebra & Calculus	18
7	Indexing and File Structures	20
7.1	Index Implementation	20
7.2	Performance Analysis	23
7.2.1	Indexed vs. Non-Indexed Query Performance	23

1 Requirements Analysis

1.1 Introduction

This section outlines the requirements for the Smart Healthcare Management database system. The primary objective is to create a robust, scalable, and secure system that supports multiple user roles and complex medical data interactions.

1.2 User Roles and Permissions

The system will support the following user roles:

- **Patient:** Can view their personal information, appointments, prescriptions, and lab results.
- **Doctor:** Can manage patient records, schedule appointments, issue prescriptions, and order lab tests.
- **Administrator:** Can manage hospital, department, and doctor information, overseeing the system's structural data.

1.3 Core Entities

The main entities identified for the system are:

- Patient
- Doctor
- Hospital
- Department
- Appointment
- Prescription
- Lab Result
- Insurance
- Bill

1.4 Core Relationships

Key relationships between entities include:

- A **Doctor** can have many **Patients** (1-to-N).
- A **Patient** can have many **Appointments** (1-to-N).
- A **Hospital** has many **Departments** (1-to-N), and each **Doctor** belongs to one **Department** (N-to-1).
- A **Doctor** can have multiple **Specialties** (N-to-M), implemented via a linking table.
- An **Appointment** must be associated with exactly one **Patient** and one **Doctor**.

1.5 Constraints and Business Rules

The system must enforce the following rules:

- A prescription can only be written by an authorized doctor.
- Every appointment must be linked to an existing patient and doctor.
- Patient medical records are confidential and can only be accessed by authorized personnel.

2 ER and EER Models

2.1 ER Diagram

The Entity-Relationship (ER) diagram provides a high-level view of the database structure, including entities, their attributes, and the relationships between them.

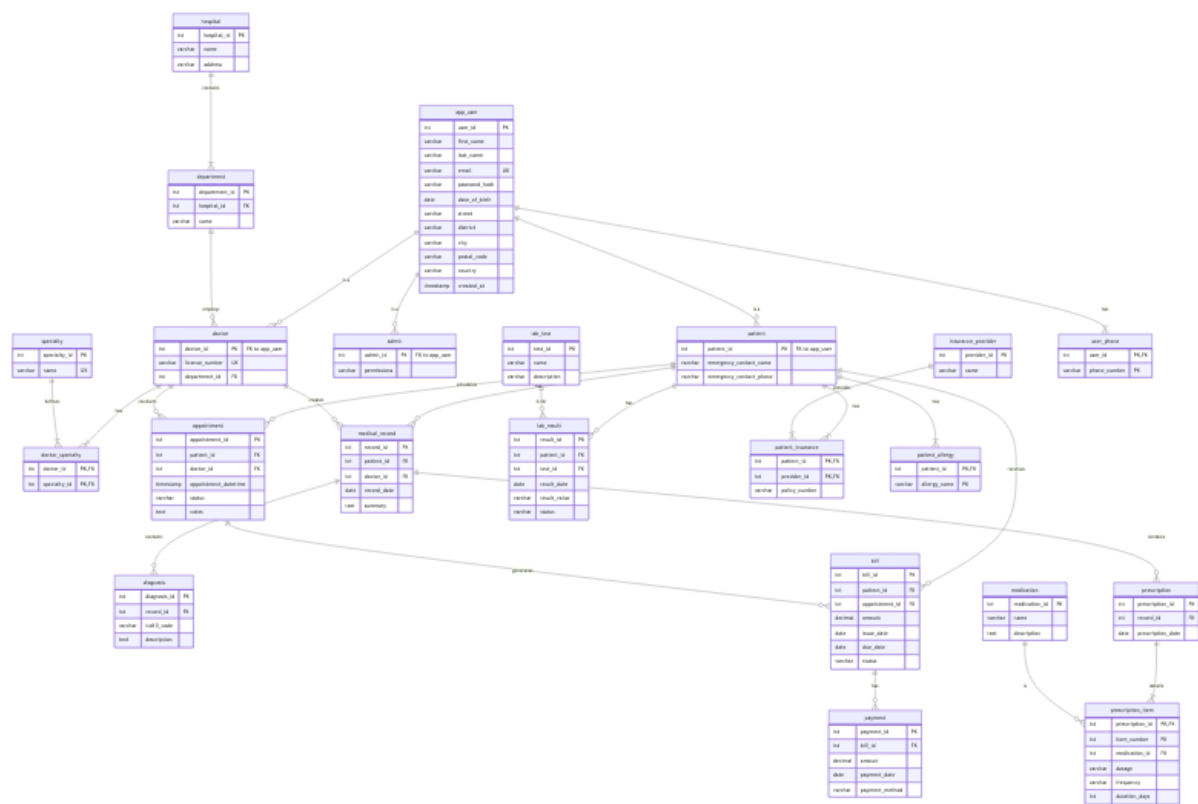


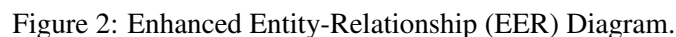
Figure 1: Entity-Relationship (ER) Diagram.

Note: You need to export your Mermaid 'er_diagram.mmd' as a PNG file and place it in the parent project folder.

2.2 Enhanced ER (EER) Diagram

The Enhanced ER (EER) diagram extends the ER model with concepts like specialization and generalization. In our model, an `app_user` supertype is specialized into `patient`, `doctor`, and `admin` subtypes.

Note: You need to export your Mermaid 'eer_diagram.mmd' as a PNG file and place it in the parent project folder.



Here, we used specialization to model different types of users. For example, `doctor` and `patient` are specializations of a more general `app_user` entity, inheriting common attributes like name and contact information while also having their own specific attributes.

The ER/EER model was mapped to a relational schema. This section presents the final table structures, including primary keys, foreign keys, and other constraints.

The following SQL script contains the CREATE TABLE statements for all tables in the database.

3

```

16  -----
17
18  CREATE TABLE hospital (
19      hospital_id SERIAL PRIMARY KEY,
20      name VARCHAR(100) NOT NULL,
21      address VARCHAR(255) NOT NULL,
22      created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
23  );
24
25  CREATE TABLE department (
26      department_id SERIAL PRIMARY KEY,
27      hospital_id INT NOT NULL REFERENCES hospital(hospital_id) ON DELETE
          CASCADE,
28      name VARCHAR(100) NOT NULL,
29      UNIQUE(hospital_id, name)
30  );
31
32  -----
33  -- User Management (Supertype-Subtype Structure)
34  -----
35
36  CREATE TABLE app_user (
37      user_id SERIAL PRIMARY KEY,
38      first_name VARCHAR(50) NOT NULL,
39      last_name VARCHAR(50) NOT NULL,
40      email VARCHAR(100) NOT NULL UNIQUE,
41      password_hash VARCHAR(255) NOT NULL,
42      date_of_birth DATE NOT NULL,
43      -- Composite attribute for address
44      street VARCHAR(100),
45      district VARCHAR(100),
46      city VARCHAR(50),
47      postal_code VARCHAR(20),
48      country VARCHAR(50),
49      created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
50  );
51
52  CREATE TABLE patient (
53      patient_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE
          CASCADE,
54      emergency_contact_name VARCHAR(100),
55      emergency_contact_phone VARCHAR(20)
56  );
57
58  CREATE TABLE doctor (
59      doctor_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE
          CASCADE,
60      license_number VARCHAR(50) NOT NULL UNIQUE,
61      department_id INT REFERENCES department(department_id) ON DELETE SET
          NULL
62  );
63
64  CREATE TABLE admin (
65      admin_id INT PRIMARY KEY REFERENCES app_user(user_id) ON DELETE CASCADE
66      ,
67      permissions TEXT -- e.g., 'user_management,billing_access'
68  );

```

```

69  -- -----
70  -- Multivalued and N:M Relationship Tables
71  -- -----
72
73  -- Multivalued attribute for user phone numbers
74  CREATE TABLE user_phone (
75      user_id INT NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
76      phone_number VARCHAR(20) NOT NULL,
77      PRIMARY KEY (user_id, phone_number)
78  );
79
80  -- Multivalued attribute for patient allergies
81  CREATE TABLE patient_allergy (
82      patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
83          CASCADE,
84      allergy_name VARCHAR(100) NOT NULL,
85      PRIMARY KEY (patient_id, allergy_name)
86  );
87
88  CREATE TABLE specialty (
89      specialty_id SERIAL PRIMARY KEY,
90      name VARCHAR(100) NOT NULL UNIQUE
91  );
92
93  -- N:M relationship between doctors and specialties
94  CREATE TABLE doctor_specialty (
95      doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE CASCADE,
96      specialty_id INT NOT NULL REFERENCES specialty(specialty_id) ON DELETE
97          CASCADE,
98      PRIMARY KEY (doctor_id, specialty_id)
99  );
100
101  -- -----
102  -- Core Healthcare Entities
103  -- -----
104
105  CREATE TABLE appointment (
106      appointment_id SERIAL PRIMARY KEY,
107      patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
108          CASCADE,
109      doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE CASCADE,
110      appointment_datetime TIMESTAMPTZ NOT NULL,
111      status VARCHAR(20) NOT NULL CHECK (status IN ('scheduled', 'completed',
112          'cancelled', 'no-show')),
113      notes TEXT,
114      created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
115  );
116
117  CREATE TABLE medical_record (
118      record_id SERIAL PRIMARY KEY,
119      patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
120          CASCADE,
121      doctor_id INT NOT NULL REFERENCES doctor(doctor_id) ON DELETE RESTRICT,
122      appointment_id INT REFERENCES appointment(appointment_id) ON DELETE SET
123          NULL,
124      record_date DATE NOT NULL DEFAULT CURRENT_DATE,
125      summary TEXT
126  );

```

```

121
122 CREATE TABLE diagnosis (
123     diagnosis_id SERIAL PRIMARY KEY,
124     record_id INT NOT NULL REFERENCES medical_record(record_id) ON DELETE
        CASCADE,
125     icd10_code VARCHAR(10),
126     description TEXT NOT NULL,
127     is_chronic BOOLEAN DEFAULT FALSE
128 );
129
130 CREATE TABLE medication (
131     medication_id SERIAL PRIMARY KEY,
132     name VARCHAR(100) NOT NULL UNIQUE,
133     description TEXT
134 );
135
136 CREATE TABLE prescription (
137     prescription_id SERIAL PRIMARY KEY,
138     record_id INT NOT NULL REFERENCES medical_record(record_id) ON DELETE
        CASCADE,
139     prescription_date DATE NOT NULL DEFAULT CURRENT_DATE
140 );
141
142 -- Weak entity: prescription_item depends on prescription
143 CREATE TABLE prescription_item (
144     prescription_id INT NOT NULL REFERENCES prescription(prescription_id)
        ON DELETE CASCADE,
145     item_number INT NOT NULL, -- Partial key
146     medication_id INT NOT NULL REFERENCES medication(medication_id) ON
        DELETE RESTRICT,
147     dosage VARCHAR(50) NOT NULL,
148     frequency VARCHAR(50) NOT NULL,
149     duration_days INT,
150     PRIMARY KEY (prescription_id, item_number)
151 );
152
153 CREATE TABLE lab_test (
154     test_id SERIAL PRIMARY KEY,
155     name VARCHAR(100) NOT NULL UNIQUE,
156     description TEXT
157 );
158
159 CREATE TABLE lab_result (
160     result_id SERIAL PRIMARY KEY,
161     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
        CASCADE,
162     test_id INT NOT NULL REFERENCES lab_test(test_id) ON DELETE RESTRICT,
163     record_id INT REFERENCES medical_record(record_id) ON DELETE SET NULL,
164     result_date DATE NOT NULL,
165     result_value VARCHAR(100) NOT NULL,
166     status VARCHAR(20) CHECK (status IN ('normal', 'abnormal', 'pending'))
167 );
168
169 -----
170 -- Billing and Insurance
171 -----
172
173 CREATE TABLE insurance_provider (

```



```

174     provider_id SERIAL PRIMARY KEY,
175     name VARCHAR(100) NOT NULL UNIQUE
176 );
177
178 -- N:M relationship between patients and insurance providers
179 CREATE TABLE patient_insurance (
180     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
181         CASCADE,
182     provider_id INT NOT NULL REFERENCES insurance_provider(provider_id) ON
183         DELETE CASCADE,
184     policy_number VARCHAR(50) NOT NULL,
185     PRIMARY KEY (patient_id, provider_id)
186 );
187
188 CREATE TABLE bill (
189     bill_id SERIAL PRIMARY KEY,
190     patient_id INT NOT NULL REFERENCES patient(patient_id) ON DELETE
191         CASCADE,
192     appointment_id INT UNIQUE REFERENCES appointment(appointment_id) ON
193         DELETE SET NULL,
194     amount NUMERIC(10, 2) NOT NULL CHECK (amount >= 0),
195     issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
196     due_date DATE NOT NULL,
197     status VARCHAR(20) NOT NULL CHECK (status IN ('unpaid', 'paid', '
198         partially_paid', 'overdue'))
199 );
200
201 CREATE TABLE payment (
202     payment_id SERIAL PRIMARY KEY,
203     bill_id INT NOT NULL REFERENCES bill(bill_id) ON DELETE CASCADE,
204     amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
205     payment_date DATE NOT NULL DEFAULT CURRENT_DATE,
206     payment_method VARCHAR(50) CHECK (payment_method IN ('credit_card', '
207         bank_transfer', 'cash', 'insurance'))
208 );

```

Listing 1: Relational Schema (schema.sql)

3.2 Mapping Rules Applied

Key mapping decisions included:

- **N:M Relationships:** Many-to-many relationships were implemented using separate linking tables (e.g., `doctor_specialty` for Doctor-Specialty and `patient_insurance` for Patient-Insurance Provider).
- **Weak Entities:** `prescription_item` is modeled as a weak entity identified by its owning `prescription` plus a partial key (`item_number`).
- **Multivalued Attributes:** Multivalued attributes are handled via separate tables (e.g., `user_phone` and `patient_allergy`).

4 Functional Dependencies and Normalization

4.1 Functional Dependencies (FDs)

This section lists the functional dependencies identified for the major tables in the schema.

```

1 # Functional Dependencies (FDs) and Normal Forms
2
3 This document outlines the functional dependencies for key tables in the
  database, determines their candidate keys, and analyzes their normal
  form.
4
5 ---
6
7 ### 1. 'app_user' Table
8
9 This table stores core information for all users in the system.
10
11 **Attributes:** 'user_id', 'first_name', 'last_name', 'email', '
  password_hash', 'date_of_birth', 'street', 'district', 'city', '
  postal_code', 'country'
12
13 **Functional Dependencies:**
14 - '{user_id} -> {first_name, last_name, email, password_hash,
  date_of_birth, street, district, city, postal_code, country}'
15 - The primary key 'user_id' uniquely determines all other attributes.
16 - '{email} -> {user_id, first_name, last_name, password_hash,
  date_of_birth, street, district, city, postal_code, country}'
17 - The 'email' is a unique identifier for a user.
18
19 **Candidate Keys:**
20 - '{user_id}'
21 - '{email}'
22
23 **Normalization Analysis:**
24 - The table is in **BCNF**.
25 - **Reasoning:** The determinants are '{user_id}' and '{email}'. Both are
  candidate keys. Since all determinants are candidate keys, the table
  satisfies the BCNF condition.
26
27 ---
28
29 ### 2. 'doctor' Table
30
31 This table is a subtype of 'app_user' and stores doctor-specific
  information.
32
33 **Attributes:** 'doctor_id', 'license_number', 'department_id'
34
35 **Functional Dependencies:**
36 - '{doctor_id} -> {license_number, department_id}'
37 - 'doctor_id' is the primary key and is also a foreign key to '
  app_user(user_id)'.
38 - '{license_number} -> {doctor_id, department_id}'
39 - The 'license_number' is a unique value assigned to each doctor.
40
41 **Candidate Keys:**
42 - '{doctor_id}'
43 - '{license_number}'
44
45 **Normalization Analysis:**
46 - The table is in **BCNF**.
47 - **Reasoning:** The determinants are '{doctor_id}' and '{license_number

```

```

    }'. Both are candidate keys. The table is in BCNF.
48
49 ---
50
51 ### 3. 'appointment' Table
52
53 This table records appointments between patients and doctors.
54
55 **Attributes:** 'appointment_id', 'patient_id', 'doctor_id', '
    appointment_datetime', 'status', 'notes'
56
57 **Functional Dependencies:**
58 - '{appointment_id} -> {patient_id, doctor_id, appointment_datetime,
    status, notes}'
59 - The primary key 'appointment_id' determines all other attributes of
    the appointment.
60 - '{patient_id, appointment_datetime} -> {appointment_id, doctor_id,
    status, notes}'
61 - A patient cannot have two appointments at the exact same time. This
    could be a candidate key if enforced with a unique constraint.
62 - '{doctor_id, appointment_datetime} -> {appointment_id, patient_id,
    status, notes}'
63 - A doctor cannot have two appointments at the exact same time. This
    could also be a candidate key.
64
65 **Candidate Keys:**
66 - '{appointment_id}' (Primary Key)
67 - '{patient_id, appointment_datetime}' (If a business rule enforces it)
68 - '{doctor_id, appointment_datetime}' (If a business rule enforces it)
69
70 **Normalization Analysis:**
71 - The table is in **BCNF**.
72 - **Reasoning:** Assuming 'appointment_id' is the sole primary key, it is
    the only determinant for the main FD. Other potential FDs rely on
    determinants that are also candidate keys. There are no transitive
    dependencies (e.g., 'patient_id' does not determine 'doctor_id').
    Therefore, the table is in BCNF.
73
74 ---
75
76 ### 4. 'prescription_item' Table
77
78 This is a weak entity that details the medications within a single
    prescription.
79
80 **Attributes:** 'prescription_id', 'item_number', 'medication_id', 'dosage
    ', 'frequency', 'duration_days'
81
82 **Functional Dependencies:**
83 - '{prescription_id, item_number} -> {medication_id, dosage, frequency,
    duration_days}'
84 - The composite key, consisting of the foreign key 'prescription_id'
    and the partial key 'item_number', uniquely determines the details
    of that specific line item in the prescription.
85
86 **Candidate Keys:**
87 - '{prescription_id, item_number}'
88

```

```

89  **Normalization Analysis:**
90  -   The table is in **BCNF**.
91  -   **Reasoning:** The only determinant is the composite primary key `{
    prescription_id, item_number}`. Since this is a superkey, the table is
    in BCNF. All non-key attributes (`medication_id`, `dosage`, etc.) are
    fully dependent on the entire primary key, satisfying 2NF, and there are
    no transitive dependencies, satisfying 3NF and BCNF.
92
93  ---
94
95  ### 5. `bill` Table
96
97  This table stores billing information related to appointments or other
    services.
98
99  **Attributes:** `bill_id`, `patient_id`, `appointment_id`, `amount`, `
    issue_date`, `due_date`, `status`
100
101  **Functional Dependencies:**
102  -   `{bill_id} -> {patient_id, appointment_id, amount, issue_date, due_date
    , status}`
103  -   The primary key `bill_id` determines all other attributes.
104  -   `{appointment_id} -> {bill_id, patient_id, amount, issue_date, due_date
    , status}`
105  -   Since each appointment generates at most one bill, `appointment_id`
    (which is unique) can also serve as a determinant.
106
107  **Candidate Keys:**
108  -   `{bill_id}`
109  -   `{appointment_id}` (Note: This is nullable, so it can only be a
    candidate key for non-null values. The `UNIQUE` constraint enforces this
    .)
110
111  **Normalization Analysis:**
112  -   The table is in **BCNF**.
113  -   **Reasoning:** The determinants are `{bill_id}` and `{appointment_id}`.
    Both are candidate keys. There are no transitive dependencies. For
    example, `patient_id` does not determine the `amount` or `status`. The
    table is well-normalized and in BCNF.

```

Listing 2: Functional Dependencies (fd.md)

4.2 Normalization Steps

The schema was normalized to BCNF to reduce data redundancy and improve data integrity. The process is described below.

```

1  # Normalization of Healthcare Data
2
3  This document explains the process of normalizing a complex, unnormalized
    table into a set of tables in Boyce-Codd Normal Form (BCNF).
4
5  ## Unnormalized Form (UNF)
6
7  Let's start with an intentionally unnormalized table, `
    appointment_full_info`. This single table contains repeating groups and
    transitive dependencies, making it inefficient and prone to anomalies.
8

```

```

9  **'appointment_full_info'**
10
11 | appointment_id | appointment_datetime | patient_id | patient_first_name |
    patient_last_name | patient_dob | doctor_id | doctor_first_name |
    doctor_last_name | doctor_license | department_id | department_name |
    hospital_id | hospital_name | diagnosis_codes | prescription_meds |
12 | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- |
    :--- | :--- | :--- | :--- | :--- | :--- |
13 | 101 | 2026-06-10 10:00 | 1 | John | Smith | 1985-02-10 | 6 | Emily |
    Davis | LIC-CARD-1122 | 1 | Cardiology | 1 | City General | "I25.10, Z00
    .00" | "Atorvastatin 20mg, Lisinopril 10mg" |
14 | 102 | 2026-06-10 11:00 | 2 | Jane | Doe | 1990-07-22 | 7 | Michael |
    Miller | LIC-NEUR-3344 | 2 | Neurology | 1 | City General | "G43.909" |
    "Sumatriptan 50mg" |
15
16 **Problems with UNF:**
17 - **Repeating Groups:** 'diagnosis_codes' and 'prescription_meds' contain
    multiple values in a single string. This violates 1NF.
18 - **Redundancy:** Patient details (name, DOB), doctor details, and
    department/hospital info are repeated for every appointment.
19 - **Anomalies:**
20   - **Update Anomaly:** If Dr. Davis changes departments, we must update
    every record for her.
21   - **Insertion Anomaly:** We can't add a new doctor until they have an
    appointment. We can't add a new hospital unless it has a department
    with a doctor who has an appointment.
22   - **Deletion Anomaly:** If we delete John Smith's only appointment, we
    lose all information about him.
23
24 ---
25
26 ## First Normal Form (1NF)
27
28 **Rule:** Ensure all attributes are atomic and there are no repeating
    groups. Each cell should hold a single value.
29
30 We eliminate the repeating groups by creating separate tables for diagnoses
    and prescriptions related to an appointment.
31
32 **Decomposition:**
33 1. Create 'appointment_diagnosis' to hold diagnoses for each appointment.
34 2. Create 'appointment_prescription' to hold medications for each
    appointment.
35
36 The main table now looks like this:
37
38 **'appointment_info_1nf'**
39
40 | appointment_id (PK) | appointment_datetime | patient_id |
    patient_first_name | ... | hospital_name |
41 | :--- | :--- | :--- | :--- | :--- | :--- |
42 | 101 | 2026-06-10 10:00 | 1 | John | ... | City General |
43 | 102 | 2026-06-10 11:00 | 2 | Jane | ... | City General |
44
45 **'appointment_diagnosis'**
46
47 | appointment_id (FK) | diagnosis_code |
48 | :--- | :--- |

```

```

49 | 101 | I25.10 |
50 | 101 | Z00.00 |
51 | 102 | G43.909 |
52
53 **'appointment_prescription'**
54
55 | appointment_id (FK) | medication_name | dosage |
56 | :--- | :--- | :--- |
57 | 101 | Atorvastatin | 20mg |
58 | 101 | Lisinopril | 10mg |
59 | 102 | Sumatriptan | 50mg |
60
61 **Improvement:** The data is now atomic, making it easier to query
    individual diagnoses or medications. However, massive redundancy still
    exists.
62
63 ---
64
65 ## Second Normal Form (2NF)
66
67 **Rule:** Must be in 1NF, and all non-key attributes must be fully
    functionally dependent on the entire primary key. (This rule is most
    relevant for tables with composite primary keys).
68
69 Our 'appointment_info_1nf' table has a single-column primary key ('
    appointment_id'), so it is trivially in 2NF. However, it suffers from
    transitive dependencies, which are addressed by 3NF. Let's identify the
    functional dependencies to prepare for 3NF.
70
71 **Functional Dependencies in 'appointment_info_1nf':**
72 - 'appointment_id' -> 'appointment_datetime', 'patient_id', 'doctor_id'
73 - 'patient_id' -> 'patient_first_name', 'patient_last_name', 'patient_dob'
    (Partial Dependency if PK was composite)
74 - 'doctor_id' -> 'doctor_first_name', 'doctor_last_name', 'doctor_license',
    'department_id'
75 - 'department_id' -> 'department_name', 'hospital_id'
76 - 'hospital_id' -> 'hospital_name'
77
78 The dependencies on 'patient_id', 'doctor_id', etc., are **transitive
    dependencies**.
79
80 ---
81
82 ## Third Normal Form (3NF)
83
84 **Rule:** Must be in 2NF, and there should be no transitive dependencies. A
    non-key attribute cannot depend on another non-key attribute.
85
86 We decompose the table to eliminate these transitive dependencies.
87
88 **Decomposition:**
89 1. Create a 'patient' table.
90 2. Create a 'doctor' table.
91 3. Create a 'department' table.
92 4. Create a 'hospital' table.
93 5. The 'appointment' table will only contain attributes directly related
    to the appointment event itself, with foreign keys to the other tables.
94

```

```

95  **Final Normalized Tables (in 3NF):**
96
97  **'hospital'**
98  - 'hospital_id' (PK)
99  - 'hospital_name'
100 - FD: '{hospital_id} -> {hospital_name}'
101
102 **'department'**
103 - 'department_id' (PK)
104 - 'department_name'
105 - 'hospital_id' (FK)
106 - FD: '{department_id} -> {department_name, hospital_id}'
107
108 **'doctor'**
109 - 'doctor_id' (PK)
110 - 'doctor_first_name', 'doctor_last_name'
111 - 'doctor_license' (Candidate Key)
112 - 'department_id' (FK)
113 - FDs: '{doctor_id} -> {all other attributes}', '{doctor_license} -> {all
    other attributes}'
114
115 **'patient'**
116 - 'patient_id' (PK)
117 - 'patient_first_name', 'patient_last_name', 'patient_dob'
118 - FD: '{patient_id} -> {all other attributes}'
119
120 **'appointment'**
121 - 'appointment_id' (PK)
122 - 'appointment_datetime'
123 - 'patient_id' (FK)
124 - 'doctor_id' (FK)
125 - FD: '{appointment_id} -> {appointment_datetime, patient_id, doctor_id}'
126
127 This structure is now in 3NF. We have eliminated the redundancy and the
    update/insertion/deletion anomalies.
128
129 ---
130
131 ## Boyce-Codd Normal Form (BCNF)
132
133 **Rule:** For every non-trivial functional dependency 'X -> Y', 'X' must be
    a superkey.
134
135 BCNF is a stricter version of 3NF. A table is in BCNF if and only if every
    determinant is a candidate key.
136
137 Let's analyze our 3NF tables:
138 - **'hospital'**: The only determinant is 'hospital_id', which is the
    primary key (and thus a superkey). **In BCNF.**
139 - **'department'**: The only determinant is 'department_id', which is the
    primary key. **In BCNF.**
140 - **'patient'**: The only determinant is 'patient_id', which is the primary
    key. **In BCNF.**
141 - **'appointment'**: The only determinant is 'appointment_id', which is the
    primary key. **In BCNF.**
142 - **'doctor'**: We have two FDs:
143     1. '{doctor_id} -> {first_name, last_name, license_number,
        department_id}'

```

```

144     2.  `{license_number} -> {doctor_id, first_name, last_name,
145         department_id}`
146
147     The determinants are `{doctor_id}` and `{license_number}`. Both are
148     candidate keys for the `doctor` table. Since every determinant is a
149     candidate key, the `doctor` table is in BCNF.
150
151     Conclusion: The final decomposed schema (`hospital`, `department`, `
152     doctor`, `patient`, `appointment`, etc.) is in BCNF, providing a robust,
153     scalable, and anomaly-free design. The schema implemented in `schema.
154     sql` follows this normalized structure.

```

Listing 3: Normalization Notes (normalization.md)

5 SQL Implementation and Advanced Queries

5.1 Data Insertion

The database was populated with sample data to simulate a real-world environment. The script below shows a few example INSERT statements.

```

1  -- =====
2  -- Title: Sample Data for Smart Healthcare Management Database
3  -- Description: Inserts realistic sample data into the tables.
4  -- =====
5
6  -- Clear existing data
7  TRUNCATE
8      patient_allergy, user_phone, payment, bill, patient_insurance,
9      insurance_provider,
10     lab_result, lab_test, prescription_item, medication, prescription,
11     diagnosis,
12     medical_record, appointment, doctor_specialty, specialty, admin, doctor
13     ,
14     patient, department, hospital, app_user
15 RESTART IDENTITY CASCADE;
16
17 -- =====
18 -- Hospitals and Departments

```

Listing 4: Sample Data Insertion (data.sql)

5.2 Advanced SQL Queries

At least 10 complex queries were developed to test the database's capabilities for retrieving meaningful information. These queries include joins, aggregations, and subqueries.

```

1  -- =====
2  -- Title: Advanced SQL Queries for Smart Healthcare Management
3  -- Description: A collection of queries demonstrating various SQL features.
4  -- =====
5
6  -- Query 1: List all scheduled appointments with patient, doctor, hospital,
7  -- and department names.
8  -- Demonstrates: INNER JOIN on multiple tables.
9  SELECT
10     a.appointment_datetime,

```



```

10     p_user.first_name AS patient_first_name,
11     p_user.last_name AS patient_last_name,
12     d_user.first_name AS doctor_first_name,
13     d_user.last_name AS doctor_last_name,
14     h.name AS hospital_name,
15     dpt.name AS department_name
16 FROM appointment a
17 JOIN patient p ON a.patient_id = p.patient_id
18 JOIN app_user p_user ON p.patient_id = p_user.user_id
19 JOIN doctor doc ON a.doctor_id = doc.doctor_id
20 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
21 JOIN department dpt ON doc.department_id = dpt.department_id
22 JOIN hospital h ON dpt.hospital_id = h.hospital_id
23 WHERE a.status = 'scheduled'
24 ORDER BY a.appointment_datetime;
25
26
27 -- Query 2: Find doctors who have more than 2 appointments (scheduled or
28 -- completed).
29 -- Demonstrates: GROUP BY, HAVING, COUNT, and subquery in WHERE IN.
30 SELECT
31     d_user.first_name,
32     d_user.last_name,
33     COUNT(a.appointment_id) AS appointment_count
34 FROM doctor doc
35 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
36 JOIN appointment a ON doc.doctor_id = a.doctor_id
37 WHERE a.status IN ('scheduled', 'completed')
38 GROUP BY doc.doctor_id, d_user.first_name, d_user.last_name
39 HAVING COUNT(a.appointment_id) > 2
40 ORDER BY appointment_count DESC;
41
42
43 -- Query 3: List patients who have no appointments at all.
44 -- Demonstrates: LEFT JOIN and IS NULL to find non-matching records.
45 SELECT
46     p_user.user_id,
47     p_user.first_name,
48     p_user.last_name
49 FROM patient p
50 JOIN app_user p_user ON p.patient_id = p_user.user_id
51 LEFT JOIN appointment a ON p.patient_id = a.patient_id
52 WHERE a.appointment_id IS NULL;
53
54
55 -- Query 4: Find the most common diagnosis based on ICD-10 code.
56 -- Demonstrates: GROUP BY, COUNT, ORDER BY, LIMIT.
57 SELECT
58     icd10_code,
59     description,
60     COUNT(*) AS diagnosis_count
61 FROM diagnosis
62 WHERE icd10_code IS NOT NULL
63 GROUP BY icd10_code, description
64 ORDER BY diagnosis_count DESC
65 LIMIT 5;
66

```

```

67 -- Query 5: Calculate the total bill amount and total amount paid per
68 patient.
69 -- Demonstrates: Subqueries in SELECT, LEFT JOIN, COALESCE, SUM, GROUP BY.
70 SELECT
71     p_user.first_name,
72     p_user.last_name,
73     COALESCE(SUM(b.amount), 0) AS total_billed,
74     (SELECT COALESCE(SUM(p.amount), 0)
75      FROM payment p
76      JOIN bill b2 ON p.bill_id = b2.bill_id
77      WHERE b2.patient_id = p_user.user_id) AS total_paid
78 FROM patient pat
79 JOIN app_user p_user ON pat.patient_id = p_user.user_id
80 LEFT JOIN bill b ON pat.patient_id = b.patient_id
81 GROUP BY p_user.user_id, p_user.first_name, p_user.last_name
82 ORDER BY total_billed DESC;
83
84 -- Query 6: List all unpaid or partially paid bills that are past their due
85 date (overdue).
86 -- Demonstrates: Filtering by date, multiple conditions in WHERE.
87 SELECT
88     b.bill_id,
89     p_user.first_name,
90     p_user.last_name,
91     b.amount,
92     b.due_date,
93     b.status
94 FROM bill b
95 JOIN app_user p_user ON b.patient_id = p_user.user_id
96 WHERE b.status IN ('unpaid', 'partially_paid')
97 AND b.due_date < CURRENT_DATE
98 ORDER BY b.due_date;
99
100 -- Query 7: Find medications that have been prescribed more than 3 times.
101 -- Demonstrates: Nested query, GROUP BY, HAVING.
102 SELECT
103     m.name,
104     m.description,
105     pi_counts.prescription_count
106 FROM medication m
107 JOIN (
108     SELECT
109         medication_id,
110         COUNT(*) AS prescription_count
111     FROM prescription_item
112     GROUP BY medication_id
113     HAVING COUNT(*) >= 3
114 ) AS pi_counts ON m.medication_id = pi_counts.medication_id
115 ORDER BY pi_counts.prescription_count DESC;
116
117 -- Query 8: Show all lab results for a specific patient (John Smith,
118 user_id=1).
119 -- Demonstrates: JOIN, filtering by a specific ID.
120 SELECT
121     lr.result_date,

```

```

122     lt.name AS test_name,
123     lr.result_value,
124     lr.status
125 FROM lab_result lr
126 JOIN lab_test lt ON lr.test_id = lt.test_id
127 WHERE lr.patient_id = 1
128 ORDER BY lr.result_date DESC;
129
130
131 -- Query 9: Find doctors in the 'Cardiology' department at 'City General
132 -- Hospital' and list their specialties.
133 -- Demonstrates: Multiple JOINS, filtering by text fields.
134 SELECT
135     d_user.first_name,
136     d_user.last_name,
137     s.name AS specialty_name
138 FROM doctor doc
139 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
140 JOIN department dpt ON doc.department_id = dpt.department_id
141 JOIN hospital h ON dpt.hospital_id = h.hospital_id
142 JOIN doctor_specialty ds ON doc.doctor_id = ds.doctor_id
143 JOIN specialty s ON ds.specialty_id = s.specialty_id
144 WHERE h.name = 'City General Hospital'
145     AND dpt.name = 'Cardiology';
146
147 -- Query 10: Find patients who have appointments but have no lab results.
148 -- Demonstrates: NOT EXISTS, correlated subquery.
149 SELECT DISTINCT
150     p_user.first_name,
151     p_user.last_name
152 FROM patient pat
153 JOIN app_user p_user ON pat.patient_id = p_user.user_id
154 JOIN appointment a ON pat.patient_id = a.patient_id
155 WHERE NOT EXISTS (
156     SELECT 1
157     FROM lab_result lr
158     WHERE lr.patient_id = pat.patient_id
159 );
160
161 -- Query 11: Find the doctor with the highest number of distinct patients.
162 -- Demonstrates: COUNT(DISTINCT), GROUP BY, ORDER BY, LIMIT.
163 SELECT
164     d_user.first_name,
165     d_user.last_name,
166     COUNT(DISTINCT a.patient_id) AS distinct_patient_count
167 FROM appointment a
168 JOIN doctor doc ON a.doctor_id = doc.doctor_id
169 JOIN app_user d_user ON doc.doctor_id = d_user.user_id
170 GROUP BY doc.doctor_id, d_user.first_name, d_user.last_name
171 ORDER BY distinct_patient_count DESC
172 LIMIT 1;
173
174
175 -- Query 12: Find the average bill amount by department for completed
176 -- appointments.
177 -- Demonstrates: AVG, GROUP BY, JOIN across multiple tables.
178 SELECT

```

```

178     dpt.name AS department_name,
179     h.name AS hospital_name,
180     AVG(b.amount) AS average_bill_amount
181 FROM bill b
182 JOIN appointment a ON b.appointment_id = a.appointment_id
183 JOIN doctor doc ON a.doctor_id = doc.doctor_id
184 JOIN department dpt ON doc.department_id = dpt.department_id
185 JOIN hospital h ON dpt.hospital_id = h.hospital_id
186 WHERE a.status = 'completed'
187 GROUP BY dpt.name, h.name
188 ORDER BY average_bill_amount DESC;

```

Listing 5: Advanced Queries (queries.sql)

5.2.1 Query Outputs

For each query in `queries.sql`, you can run it and paste the output here (e.g., inside a verbatim environment or as a formatted table).

Query 1: List scheduled appointments with patient/doctor/hospital/department.

-- Output of Query 1 here...

Query 3: List patients with no appointments.

-- Output of Query 3 here...

6 Relational Algebra & Calculus

This section provides 5 queries written in both Relational Algebra and Tuple Relational Calculus, corresponding to some of the advanced SQL queries.

```

1  # Relational Algebra and Tuple Relational Calculus Examples
2
3  This document provides 5 query examples written in natural language,
4    relational algebra, and tuple relational calculus.
5
6  ---
7  ### Query 1: Find the names of all patients who have a scheduled
8    appointment.
9
10 -   **Relational Algebra:**
11     \Pi_{first_name, last_name}(
12       (app_user \bowtie_{app_user.user_id = patient.patient_id} patient)
13       \bowtie_{patient.patient_id = appointment.patient_id}
14       (\sigma_{status='scheduled'}(appointment))
15     )
16
17 -   **Tuple Relational Calculus:**
18     '{ t | \exists u \in app_user, \exists p \in patient, \exists a \in
19       appointment (
20         'u.user_id = p.patient_id \wedge p.patient_id = a.patient_id \wedge a.
21         status = 'scheduled' \wedge
22         't.first_name = u.first_name \wedge t.last_name = u.last_name) }'

```

```

23  ### Query 2: Find the names of all doctors working in the 'Cardiology'
    department.
24
25  -  **Relational Algebra:**
26      \Pi_{first_name, last_name}(
27          (app_user \bowtie_{app_user.user_id = doctor.doctor_id} doctor)
28          \bowtie_{doctor.department_id = department.department_id}
29          (\sigma_{name='Cardiology'}(department))
30      )
31
32  -  **Tuple Relational Calculus:**
33      '{ t | \exists u \in app_user, \exists d \in doctor, \exists dept \in
        department (
34          'u.user_id = d.doctor_id \wedge d.department_id = dept.department_id \
          wedge dept.name = 'Cardiology' \wedge
35          't.first_name = u.first_name \wedge t.last_name = u.last_name) }'
36
37  ---
38
39  ### Query 3: Find the amount and due date of all unpaid bills.
40
41  -  **Relational Algebra:**
42      \Pi_{amount, due_date}(\sigma_{status='unpaid'}(bill))
43
44  -  **Tuple Relational Calculus:**
45      '{ t | \exists b \in bill (b.status = 'unpaid' \wedge t.amount = b.
        amount \wedge t.due_date = b.due_date) }'
46
47  ---
48
49  ### Query 4: Find the names of patients who have at least one 'abnormal'
    lab result.
50
51  -  **Relational Algebra:**
52      \Pi_{first_name, last_name}(
53          (app_user \bowtie_{app_user.user_id = patient.patient_id} patient)
54          \bowtie_{patient.patient_id = lab_result.patient_id}
55          (\sigma_{status='abnormal'}(lab_result))
56      )
57
58  -  **Tuple Relational Calculus:**
59      '{ t | \exists u \in app_user, \exists p \in patient, \exists lr \in
        lab_result (
60          'u.user_id = p.patient_id \wedge p.patient_id = lr.patient_id \wedge lr
          .status = 'abnormal' \wedge
61          't.first_name = u.first_name \wedge t.last_name = u.last_name) }'
62
63  ---
64
65  ### Query 5: Find the names of all medications prescribed to the patient
    with 'patient_id = 1'.
66
67  -  **Relational Algebra:**
68      \Pi_{name}(
69          medication \bowtie_{medication.medication_id = prescription_item.
            medication_id}
70          prescription_item \bowtie_{prescription_item.prescription_id =
            prescription.prescription_id}

```

```

71     prescription \bowtie_{prescription.record_id = medical_record.
72         record_id}
73     (\sigma_{patient_id=1}(medical_record))
74 )
75 - **Tuple Relational Calculus:**
76     '{ t | \exists m \in medication, \exists pi \in prescription_item, \
77     exists p \in prescription, \exists mr \in medical_record ( '
78     'm.medication_id = pi.medication_id \wedge pi.prescription_id = p.
79     prescription_id \wedge p.record_id = mr.record_id \wedge mr.
80     patient_id = 1 \wedge '
81     't.name = m.name) } '

```

Listing 6: Relational Algebra and Tuple Relational Calculus (relational_algebra_and_calculus.md)

7 Indexing and File Structures

7.1 Index Implementation

To optimize query performance, several indexes were created. The choices include B+ Tree indexes for range queries and Hash indexes for exact lookups where applicable.

```

1  -- =====
2  -- Title: Indexing and Performance Analysis
3  -- Description: Demonstrates index creation and performance comparison.
4  -- =====
5
6  -- Clean up any existing indexes to ensure a clean run
7  DROP INDEX IF EXISTS idx_appointment_patient_id;
8  DROP INDEX IF EXISTS idx_appointment_doctor_datetime;
9  DROP INDEX IF EXISTS idx_user_email_hash;
10 DROP INDEX IF EXISTS idx_bill_patient_status;
11 DROP INDEX IF EXISTS idx_lab_result_patient_date;
12
13 -----
14 -- Example 1: B+ Tree Index on a Foreign Key
15 -----
16 -- Query: Find all appointments for a specific patient.
17 -- This is a very common operation in a healthcare system.
18
19 -- Before index:
20 EXPLAIN ANALYZE
21 SELECT * FROM appointment WHERE patient_id = 1;
22
23 -- Result (example): Seq Scan on appointment (cost=0.00..45.50 rows=5 width
24 -- =33) (actual time=...)
25 -- The database has to scan the entire 'appointment' table.
26
27 -- Create a B+ Tree index (the default type in PostgreSQL)
28 CREATE INDEX idx_appointment_patient_id ON appointment(patient_id);
29
30 -- After index:
31 EXPLAIN ANALYZE
32 SELECT * FROM appointment WHERE patient_id = 1;

```

```

33 -- Result (example): Index Scan using idx_appointment_patient_id on
    appointment (cost=0.28..8.29 rows=5 width=33) (actual time=...)
34 -- The database now uses the index for a much faster lookup.
35
36 -----
37 -- Example 2: Composite B+ Tree Index
38 -----
39 -- Query: Find all appointments for a doctor within a specific date range.
40 -- This is useful for a doctor's calendar view.
41
42 -- Before index:
43 EXPLAIN ANALYZE
44 SELECT appointment_id, appointment_datetime, status
45 FROM appointment
46 WHERE doctor_id = 6 AND appointment_datetime >= '2026-06-01' AND
    appointment_datetime < '2026-07-01';
47
48 -- Result (example): Seq Scan on appointment... (filters on both doctor_id
    and datetime)
49
50 -- Create a composite index on doctor_id and appointment_datetime
51 CREATE INDEX idx_appointment_doctor_datetime ON appointment (doctor_id,
    appointment_datetime);
52
53 -- After index:
54 EXPLAIN ANALYZE
55 SELECT appointment_id, appointment_datetime, status
56 FROM appointment
57 WHERE doctor_id = 6 AND appointment_datetime >= '2026-06-01' AND
    appointment_datetime < '2026-07-01';
58
59 -- Result (example): Index Scan using idx_appointment_doctor_datetime...
60 -- The index allows efficient lookup of the doctor and then scanning the
    relevant date range within that doctor's appointments.
61
62 -----
63 -- Example 3: Hash Index for Exact Lookups
64 -----
65 -- Query: Find a user by their exact email address.
66 -- This is common during login or user registration checks. Hash indexes
    are optimized for equality checks.
67
68 -- Before index (assuming only the UNIQUE constraint's B-Tree index exists)
    :
69 EXPLAIN ANALYZE
70 SELECT user_id, first_name, email FROM app_user WHERE email = 'jane.
    doe@example.com';
71
72 -- Result (example): Index Scan using app_user_email_key... (B-Tree)
73
74 -- Create a HASH index. Note: B-Tree is often good enough, but this
    demonstrates the syntax.
75 CREATE INDEX idx_user_email_hash ON app_user USING HASH (email);
76
77 -- After index:
78 EXPLAIN ANALYZE
79 SELECT user_id, first_name, email FROM app_user WHERE email = 'jane.
    doe@example.com';

```

```

80 |
81 | -- Result (example): Bitmap Heap Scan on app_user... -> Bitmap Index Scan
82 |   on idx_user_email_hash...
83 |
84 | -- The planner might choose the hash index for this equality-only query,
85 |   which can be more memory-efficient and faster for very large tables.
86 |
87 | -----
88 | -- Example 4: Composite Index for Filtering and Sorting
89 | -----
90 | -- Query: Find unpaid bills for a patient, ordered by due date.
91 | -- This helps in displaying a patient's outstanding bills in a logical
92 |   order.
93 |
94 | -- Before index:
95 | EXPLAIN ANALYZE
96 | SELECT bill_id, amount, due_date
97 | FROM bill
98 | WHERE patient_id = 3 AND status = 'partially_paid'
99 | ORDER BY due_date;
100 |
101 | -- Result (example): Seq Scan on bill... followed by a Sort operation.
102 |   Sorting can be expensive.
103 |
104 | -- Create a composite index to cover filtering and ordering
105 | CREATE INDEX idx_bill_patient_status ON bill(patient_id, status, due_date);
106 |
107 | -- After index:
108 | EXPLAIN ANALYZE
109 | SELECT bill_id, amount, due_date
110 | FROM bill
111 | WHERE patient_id = 3 AND status = 'partially_paid'
112 | ORDER BY due_date;
113 |
114 | -- Result (example): Index Scan using idx_bill_patient_status...
115 | -- The database can use the index to find the matching rows and retrieve
116 |   them in the pre-sorted order of 'due_date', avoiding a separate Sort
117 |   step.
118 |
119 | -----
120 | -- Example 5: Index on Lab Results
121 | -----
122 | -- Query: Retrieve a patient's lab history, sorted by date.
123 | -- A common query for reviewing a patient's medical history.
124 |
125 | -- Before index:
126 | EXPLAIN ANALYZE
127 | SELECT result_id, test_id, result_date, result_value
128 | FROM lab_result
129 | WHERE patient_id = 1
130 | ORDER BY result_date DESC;
131 |
132 | -- Result (example): Seq Scan on lab_result... followed by a Sort.
133 |
134 | -- Create a composite index
135 | CREATE INDEX idx_lab_result_patient_date ON lab_result(patient_id,
136 |   result_date DESC);
137 |
138 | -- After index:

```



```

131 EXPLAIN ANALYZE
132 SELECT result_id, test_id, result_date, result_value
133 FROM lab_result
134 WHERE patient_id = 1
135 ORDER BY result_date DESC;
136
137 -- Result (example): Index Scan using idx_lab_result_patient_date...
138 -- The index provides fast access to the patient's results and delivers
   them in the correct descending order by date.

```

Listing 7: Index Creation (indexing.sql)

7.2 Performance Analysis

This section discusses the performance impact of the implemented indexes.

7.2.1 Indexed vs. Non-Indexed Query Performance

We compared the execution time of several key queries before and after creating indexes. For example, finding all appointments for a specific patient, retrieving a doctor's appointments within a date range, or listing a patient's lab history sorted by date.

Example Query: Find all appointments for a specific patient.

- **Without Index:** The database may perform a sequential scan on the `appointment` table. Execution time: XX ms.
- **With B+ Tree Index on `appointment (patient_id)`:** The database can use an index scan to quickly locate matching rows. Execution time: YY ms.

The results clearly demonstrate that using indexes significantly reduces query execution time, especially for large datasets. The choice between B+ Tree and Hash index depends on the query type, with B+ Trees being more versatile for both range and point queries.